

Lab 3A: MCP — Connect Claude to Your Systems

Duration: 35 min **After:** MCP Deep Dive lecture (Day 3, Block 1) **Prerequisites:** Day 2 complete (toolkit plugin installed)

Objective

Configure an MCP server for your project, make a deliberate **transport choice** (stdio vs HTTP; SSE is legacy), connect and use the public Microsoft Learn HTTP server, recognize authentication as a separate server capability, and verify current ToolSearch loading behavior with `/context`.

Deliverable (toolkit chain 9 of 12)

A working `.mcp.json` in the toolkit repo exposing a `dashboard-metrics` server, plus a written transport decision.

START MARKER: In `business-dashboard`, terminal available.

Missing Your Day 2 Project? Restore It Before Lab 3A

Day 3 normally continues in the same `business-dashboard` repo used on Days 1 and 2. If you no longer have that folder, create a replacement **before the 35-minute lab begins**. This produces the minimum viable Day 2 baseline; it does not replay every earlier exercise.

In the VS Code PowerShell terminal:

```
New-Item -ItemType Directory -Force C:\Labs\business-dashboard
Set-Location C:\Labs\business-dashboard
code .
```

Paste this recovery prompt into Claude Code:

Prepare this empty folder as the minimum viable end-of-Day-2 project for our Claude Code training.

Before writing anything, inspect and report whether any of these already exist:

- files in the current project folder
- ../team-marketplace
- \$HOME/.claude/skills/dashboard-toolkit

Do not silently reuse, overwrite, or delete existing content. If anything exists, list the exact paths and ask me whether to archive, reuse, or remove it before continuing.

Build a Node.js Express project named "Business Dashboard" with:

1. package.json and appropriate dependencies, with the test script exactly "node --test" so Node discovers test/*.test.js consistently on Windows
2. server.js running on port 3100
3. src/routes/tasks.js with GET and POST /api/tasks routes using an in-memory array
4. src/routes/metrics.js with GET and POST /api/metrics routes
5. a simple dashboard page at / that displays tasks and metrics
6. basic automated tests for the routes
7. CLAUDE.md documenting:
 - Node.js and Express
 - double quotes and 2-space indentation
 - async/await
 - guard clauses and input validation
 - no credentials or secrets in source files
8. .gitignore
9. .claude/skills/dashboard-code-review/SKILL.md:
 - model-invoked
 - Read, Grep, and Glob only
 - reports RED/YELLOW/BLUE findings
10. .claude/skills/dashboard-review/SKILL.md:
 - human-invoked
 - accepts \$ARGUMENTS
 - Read, Grep, and Glob only
11. .claude/hooks/post-edit-format.sh and a PostToolUse registration in .claude/settings.json
12. a valid local dashboard-toolkit plugin package under ../team-marketplace/plugins/dashboard-toolkit containing the two skills and hook configuration
13. ../team-marketplace/.claude-plugin/marketplace.json registering dashboard-toolkit with source ./plugins/dashboard-toolkit

After creating the files:

- install dependencies
- run the tests
- start the application and verify it responds
- stop only the server process or PID you started; do not use taskkill /F /IM node.exe or any command that kills every Node process
- validate the plugin package strictly
- add ../team-marketplace as a local marketplace and install dashboard-toolkit@team-marketplace at local scope

```
- confirm the two skills and hook are present in the installed plugin
- initialize git
- commit the verified project baseline with: "chore: restore Day 2 training baseline"
- keep the sibling team-marketplace folder outside the project commit
- report the files created, test results, application URL, plugin validation/install result,
and any unresolved issue
```

Recovery acceptance gate: any pre-existing project, marketplace, or global skill state was surfaced before modification; tests pass through `node --test` ; `git status` is clean; the dashboard responds on port 3100; the local marketplace has a valid `marketplace.json` ; and `dashboard-toolkit@team-marketplace` validates and installs at local scope. If marketplace installation is blocked by policy or product access, record that limitation and complete the Lab 3D plugin inventory gate using the validated package path. Windows CRLF/LF conversion notices are normally cosmetic when `core.autocrlf` is enabled; treat them as blocking only if the diff shows actual content changes.

Phase 1: Build a Small Local Server — stdio (12 min)

Step 1 — start Claude and scaffold a minimal server:

```
Create an MCP server at mcp-server/index.js (plain Node, using @modelcontextprotocol/sdk)
named "dashboard-metrics" with two tools:
1. get_metric(metric_name) – returns mock data for: revenue (48250 USD, up 12%), customers
(342, up 8%), churn (2.1%, down 0.3%), nps (72, up 4 points). Unknown names return an error lis
ting valid options.
2. get_dashboard_summary() – returns all metrics as JSON.
Use the stdio transport. Install dependencies and verify it starts without errors.
```

Expected output marker: Claude reports the server compiles/starts silently (stdio servers wait on stdin — no output is correct).

Step 2 — register it:

```
Add this server to .mcp.json at the project root:
{
  "mcpServers": {
    "dashboard-metrics": {
      "command": "node",
      "args": ["mcp-server/index.js"]
    }
  }
}
```

Step 3 — start a new Claude Code instance and verify:

The VS Code extension discovers project MCP configuration when a Claude Code instance starts. The current instance may not refresh after `.mcp.json` is created, and `/clear` clears conversation context but does not reliably reload the MCP server inventory.

1. Open a new **Claude Code** tab/instance in the same `business-dashboard` project.
2. Open the `/` menu and select **MCP servers**.
3. Under **Project**, confirm `dashboard-metrics` appears as **Connected**.
4. In the new instance, send:

```
What MCP tools do you have available? List them.
```

Expected output marker: the MCP servers menu shows `Project (1)` with `dashboard-metrics` connected, and Claude lists `get_metric` and `get_dashboard_summary` under that server.

Step 4 — use it:

```
What's our current revenue? Use the dashboard metrics tool.
```

Expected output marker: Claude calls `get_metric` and reports 48,250 USD, up 12%. Approve the tool call when prompted.

```
Using the dashboard summary tool, add a "Live KPIs" section to the dashboard UI with the real numbers baked in.
```

That's the power move: MCP data flowing straight into generated code.

Phase 2: Transport Choice (5 min)

Three transports — pick deliberately:

Transport	When	Auth
stdio	Local tools, your machine, child process — what you just built	none needed
HTTP	Remote/shared servers (team services, SaaS)	In VS Code, use <code>/mcp</code> and choose Authenticate when OAuth is supported; otherwise use the approved server auth method
SSE	Legacy — migrate to HTTP	—

Run this as a prompt in Claude Code and save the decision as lab evidence:

```
For each scenario below, choose stdio or Streamable HTTP and explain why:  
1. A local SQLite inspector  
2. Our team's shared Jira bridge  
3. A vendor's hosted analytics API  
  
Return a table with columns: Scenario | Transport | Why | Expected authentication.  
Save the same table to labs/3a-transport-decision.md. Do not change any MCP configuration.
```

Expected output marker: `labs/3a-transport-decision.md` exists; local child process → stdio; shared/hosted service → Streamable HTTP. The authentication column distinguishes transport from the server's separate auth requirements.

Phase 3: Remote HTTP Server + Authentication Decision (10 min)

Step 1 — add the Microsoft Learn MCP server. Exit Claude first and run this in the VS Code terminal:

```
claude mcp add --transport http ms-learn https://learn.microsoft.com/api/mcp
```

With no `--scope` flag, Claude Code stores this as **local scope** for you in this project. It is private configuration in `~/.claude.json`, not the shared `.mcp.json` you created for `dashboard-metrics`.

Step 2 — verify the no-auth path in VS Code:

- 1. Open a new Claude Code tab/instance in `business-dashboard`.
- 2. Open the `/` menu and select **MCP servers**.

3. Select `ms-learn` and confirm it shows **Connected**.
4. Confirm that it connected without a browser sign-in or consent flow. Microsoft Learn is a public, no-auth server. The MCP details panel may still show generic lifecycle controls such as **Reconnect** or **Clear authentication**; those controls do not mean this server required learner authentication.

Expected output marker: `ms-learn` connects without a browser consent flow and exposes Microsoft Learn documentation tools.

Step 3 — use the remote server:

Use the Microsoft Learn MCP tools to find the official Microsoft documentation explaining how MCP servers are configured in VS Code. Return the page title, URL, and three setup points. Tell me which MCP tool you called.

Expected output marker: Claude calls one of the `ms-learn` tools and returns a Microsoft Learn URL. This server requires no authentication. For a different remote server that advertises OAuth, the VS Code path is **MCP servers** → **select server** → **Authenticate**; that comparison is knowledge for later use, not an additional server learners must configure in this lab.

Step 4 — verify:

```
claude mcp list
```

Expected output marker: `dashboard-metrics` and `ms-learn` are listed with transport and connection status. No instructor-provided URL or additional account is required.

Phase 4: ToolSearch and Context Audit (8 min)

Back in Claude (`claude`):

```
/context
```

Current Claude Code enables MCP ToolSearch automatically by default. Initially, only tool names and server instructions load; relevant tool schemas enter context after discovery or use. Note any server configured with `alwaysLoad`, any provider limitation, or any gateway that disables tool references.

```
/doctor
```

Expected output marker: `/context` shows the actual session rather than an assumed per-server tax. (`/doctor` also flags unused servers, but its report is long and machine-specific — `/context` is the number that matters here.) If `ms-learn` is not needed after the lab, remove it:

```
claude mcp remove ms-learn
```

Commit:

```
Commit .mcp.json and mcp-server/ with "feat: add dashboard-metrics MCP server (stdio)"
```

FINISH MARKER — Success Checklist

- [] dashboard-metrics server responds to both tools from Claude
- [] `.mcp.json` committed at project root
- [] `labs/3a-transport-decision.md` saved with transport, rationale, and expected authentication for all 3 scenarios
- [] `ms-learn` connected and answered one documentation question; learner can state that OAuth-capable servers use **MCP servers → Authenticate**, but no OAuth server is required for this lab
- [] `claude mcp list` shows servers with auth status
- [] ToolSearch behavior checked with `/context` ; `alwaysLoad` or provider exceptions recorded; unused servers pruned

OPTIONAL CHALLENGE — Inspect the Raw MCP Contract (8 min)

Start this only after every item in the Success Checklist is complete. The lecture demonstrated what MCP Inspector does; this challenge has you use it to compare a raw server response with Claude's synthesized answer. It is not required for later labs.

Step 1 — launch Inspector directly against the public server:

Stop any Inspector instance already running with `Ctrl+C`, then run:

```
npx @modelcontextprotocol/inspector --web --transport http --server-url https://learn.microsoft.com/api/mcp
```

Open the one-time browser URL printed in the terminal. Inspector v2 opens this ad-hoc server as a **read-only session**; that banner is expected and only means the server entry cannot be edited or saved from this session. Turn on **Connected** for the Microsoft Learn server if it is not already connected. No Microsoft account, authentication, catalog editing, or instructor-provided value is required. If you instead see the sample `node mcp-server/index.js` card, stop Inspector and rerun the exact command above. If your installed Node version is older than 22.19, skip the challenge and record `Inspector unavailable on this machine`; do not spend core lab time upgrading Node.

Step 2 — inspect before calling:

1. Open **Tools** and list the server's tools.
2. Select `microsoft_docs_search`.
3. Read its description, input schema, and annotations.
4. Identify the required input field and whether the tool declares itself read-only.

Step 3 — call the tool manually:

Use this query as the `query` argument:

```
configure MCP servers in Visual Studio Code
```

Run the tool and inspect the raw result. Do not ask Claude to interpret it yet.

Step 4 — record your observation:

Create `labs/3a-inspector-observation.md` in VS Code with these completed fields:


```
# MCP Inspector Observation
- Tool called:
- Required input field:
- Read-only annotation:
- First result: [title](URL)
- Raw result vs. Claude's answer:
- If Inspector succeeds but Claude fails, inspect next:
```

Done when: every field contains an observed value from Inspector and the final field identifies the client/tool-selection path before the server. Close Inspector with `Ctrl+C` in its terminal.

If Stuck

Problem	Recovery
Tools don't appear	Confirm <code>.mcp.json</code> is at the project ROOT (not <code>.claude/</code>), then open a new Claude Code tab/instance in that project. <code>/clear</code> alone may not refresh the MCP inventory. Open MCP servers and confirm <code>dashboard-metrics</code> is connected.
Server errors on start	<code>!node mcp-server/index.js</code> and paste the error to Claude: "Fix this MCP server startup error"
Browser authentication does not complete	Return to <code>/mcp</code> , confirm the server still needs authentication, check the corporate proxy, and use the demo fallback if blocked
Wrong data returned	Mock data is whatever Claude generated — verify structure (name/value/trend), not exact numbers
Too many tools in context	Confirm ToolSearch is enabled and supported; inspect <code>alwaysLoad</code> ; disable unused servers

Debrief Questions

1. When is stdio the wrong choice even for a tool you wrote yourself?
2. What does the `/mcp` OAuth flow give a team that pasting API keys into config files does not?
3. What loads before tool use, what can force full schemas up front, and how do you verify the actual session?